



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

Ingeniería en Informática

Unidad Curricular: ingeniería de software II

GESTIÓN DEL VALOR, TRANSICIÓN A OPERACIONES Y MEJORA CONTINUA

Profesor:
Márquez Félix

Bachiller:
Alondra Moreno CI: 30.331.870

Samuel Cortez CI: 27.158.842

Carlos Hernández CI: 27.975.753

Eduardo Quintero CI: 32.004.965

Puerto Ordaz Septiembre de 2026

Introducción

Cerrar un proyecto de software suele ser uno de los mayores dolores de cabeza para cualquier organización. Históricamente, nos acostumbramos a pensar que el éxito se resumía en cumplir con el cronograma y el presupuesto: entregabas el código, armabas un par de manuales estáticos y el equipo se lavaba las manos. Sin embargo, hoy en día sabemos que eso ya no es suficiente. De nada sirve terminar a tiempo y dentro del presupuesto si, cuando el sistema sale a producción, no aporta el valor real que el negocio esperaba o resulta ser un dolor mantenerlo operando. Este documento, enfocado en las Unidades VII y VIII de Ingeniería de Software II, viene a cuestionar precisamente esa vieja forma de trabajar. A lo largo del informe, vamos a analizar cómo los métodos tradicionales de seguimiento a veces nos engañan. Nos dan una falsa "ilusión de avance" solo porque los números presupuestarios cuadran, cuando en realidad necesitamos combinar ese control con métricas ágiles modernas y de alto rendimiento (como el modelo DORA), las cuales miden de verdad la velocidad, la estabilidad y la calidad real de lo que estamos entregando.

En el fondo, el gran objetivo de este informe es impulsar un cambio profundo en nuestra mentalidad. Queremos dejar atrás la idea de que el software es un simple "proyecto" temporal que nos quitamos de encima el día del cierre, para empezar a construir y gestionar "productos" vivos, que se sostienen solos, evolucionan y mejoran continuamente junto con la empresa.

CASO 1: LA CONTRADICCIÓN ENTRE EL CONTROL TRADICIONAL Y EL VALOR REAL

1. Análisis de la Contradicción: Métricas de Control Tradicional vs. Realidad del Negocio

En el caso planteado, el Director de Proyecto reporta indicadores óptimos según la Gestión del Valor Ganado (EVM): un Índice de Desempeño del Costo (CPI) de 1.05 (gasto por debajo del presupuesto) y un Índice de Desempeño del Cronograma (SPI) de 1.02 (adelantado según el cronograma sintético de Gantt). Sin embargo, el área de negocio denuncia retrasos graves en la llegada de funcionalidades y alta tasa de incidentes en producción.

¿Por qué ocurre esta aparente paradoja?

La contradicción se debe a que las métricas tradicionales de gestión de proyectos (CPI y SPI) miden únicamente la 'conformidad con el plan' y la 'eficiencia en la producción de código' (Output), pero ignoran por completo la 'entrega efectiva de valor al usuario final' (Outcome).

- **1. La Ilusión del Avance en Papel (Output vs. Outcome):** El CPI y el SPI registran avance cuando los desarrolladores completan tareas en repositorios locales o entornos de pruebas. Sin embargo, en la ingeniería de software moderna, el código que no está desplegado en producción tiene un Valor Real equivalente a CERO y representa un inventario no vendido (Work in Process - WIP) que genera costos de mantenimiento y riesgo acumulado.
- **2. Acumulación de Lotes Grandes y Tiempo de Retención:** El proyecto opera en 'lotes gigantescos' (Big Batches). Aunque la programación de las tareas vaya 'adelantada' en el diagrama de Gantt, si los cambios se retienen durante semanas en ramas de integración antes de liberarse, el tiempo total desde la idea hasta el usuario se dispara.

- **3. Riesgo Oculto y Fallas de Integración Tardia:** Al acumular grandes volúmenes de código sin probar en condiciones reales de usuarios y producción, se genera una 'explosión de integración'. Cuando finalmente se realiza el despliegue, surgen fallas masivas e incidentes urgentes, demostrando que el avance reportado por el SPI era una falsa sensación de seguridad.

2. Métricas de Flujo y DORA para un Diagnóstico Real

Para reemplazar la falsa seguridad del CPI/SPI y diagnosticar la raíz del problema, el equipo debe implementar de inmediato métricas empíricas de desempeño en ingeniería de software:

- **Métrica 1: Lead Time for Changes (Tiempo de Entrega de Cambios - DORA):** Mide el tiempo total transcurrido desde que un cambio de código es ingresado al repositorio (commit) hasta que se encuentra funcionando de forma segura en el entorno de producción a disposición de los usuarios. Esta métrica revela de inmediato en qué etapas del pipeline (pruebas, aprobaciones, despliegue) se producen los cuellos de botella que desesperan al negocio.
- **Métrica 2: Deployment Frequency (Frecuencia de Despliegue - DORA):** Mide la frecuencia con la que la organización libera código de forma exitosa a producción (diaria, semanal, mensual). Una alta frecuencia de despliegue obliga a reducir el tamaño de los lotes de cambios, lo que acelera el aprendizaje y reduce drásticamente el riesgo de cada liberación.
- **Métrica 3: Change Failure Rate (Tasa de Fallas en Cambios - DORA):** Mide el porcentaje de despliegues en producción que causan fallas degradantes, incidentes urgentes o requieren parches inmediatos. Si esta métrica es alta (ej. > 20%), diagnostica de inmediato que el equipo está sacrificando la calidad y las pruebas para inflar artificialmente el SPI en el Gantt.

- **Métrica 4: Time to Restore Service (Tiempo de Restauración del Servicio - DORA):**

Mide la duración exacta de la recuperación del sistema tras presentarse un incidente, degradación o interrupción en producción. En el caso planteado, dado que el área de negocio reporta incidentes que requieren "correcciones urgentes", esta métrica es vital. Un equipo de alto rendimiento no es aquel que nunca falla, sino el que logra minimizar este tiempo de restauración a minutos, garantizando la resiliencia del sistema y la confianza del cliente.

3. La Falacia del EVM en Trabajos de Conocimiento:

Las métricas tradicionales de la Gestión del Valor Ganado, como el CPI y el SPI, fueron diseñadas originalmente para industrias predictivas como la construcción civil y la manufactura. En esos sectores, los requisitos son rígidos y el avance es físico y lineal (por ejemplo, metros de concreto vertido). Sin embargo, el desarrollo de software es un trabajo de conocimiento, altamente creativo e iterativo. Aplicar el EVM de forma aislada en este contexto genera un incentivo perverso: un equipo puede programar apresuradamente y escribir miles de líneas de código para inflar artificialmente el SPI, mostrando un falso "adelanto" en el diagrama de Gantt. La trampa radica en que esta velocidad descontrolada suele lograrse sacrificando las pruebas y la arquitectura, inyectando una deuda técnica masiva que, a largo plazo, destruye la mantenibilidad del producto y provoca exactamente los incidentes críticos en producción que el negocio está denunciando.

4. Contraste de Paradigmas de Medición

Criterio de Evaluación	Gestión Tradicional (EVM - Gantt)	Gestión Moderna (DORA - Flujo)
Foco de la Métrica	Esfuerzo, presupuesto y cumplimiento del plan inicial.	Entrega de valor al usuario y estabilidad del sistema.
Unidad de Medida	Tareas completadas (Output).	Funcionalidades usables en producción (Outcome).
Gestión del Riesgo	Se asume que cumplir el cronograma reduce el riesgo.	Se mitiga el riesgo mediante entregas en lotes pequeños.

Visibilidad de Errores	Tardía (se descubren en la fase de integración final).	Inmediata (a través de CI/CD y pruebas automatizadas).
Indicador Principal	SPI (Schedule Performance Index) y CPI.	Frecuencia de Despliegue y Tiempo de Entrega (Lead Time).

CASO 2: EL RIESGO DE 'ENTREGAR Y ABANDONAR' EN LA TRANSICIÓN A OPERACIONES

1. El Riesgo Crítico del Enfoque 'Entregar y Abandonar' (Throwing Code Over the Wall)

La intención del equipo de desarrollo de disolverse inmediatamente tras completar la implementación, limitándose a entregar un manual impreso al equipo de soporte, representa un antipatrón clásico en la ingeniería de software conocido como 'Lanzar el paquete sobre el muro' (Throwing over the wall).

Consecuencias y Riesgos Críticos para la Organización:

- 1. Ruptura de la Responsabilidad de Producto (You Build It, You Run It):** Werner Vogels (CTO de Amazon) sintetizó la filosofía DevOps moderna con la premisa: 'Tú lo construyes, tú lo ejecutas' (You build it, you run it). Cuando los desarrolladores saben que no tendrán que atender los incidentes nocturnos en producción, se genera un incentivo perverso para ignorar la mantenibilidad, la observabilidad y la deuda técnica, transfiriendo todo el dolor operativo a terceros.
- 2. Pérdida de Conocimiento Institucional y Manuales Obsoletos:** Un manual de usuario en PDF o Word queda desactualizado el primer día de operación. El equipo de soporte técnico se ve inundado de incidentes complejos que no sabe cómo resolver porque carece del contexto de diseño, la arquitectura y las decisiones de código tomadas durante el desarrollo.

- **3. Explosión del Costo Total de Propiedad (TCO):** El costo de mantener y reparar un sistema mal documentado sin soporte de los autores originales supera con frecuencia el costo inicial de desarrollo. Esto dispara el Costo Total de Propiedad (Total Cost of Ownership - TCO) y paraliza futuras evoluciones del software.

2. Tres Acciones Obligatorias para una Transición Exitosa a Producto

El Director de Proyecto debe rechazar el cierre administrativo apresurado y ejecutar obligatoriamente las siguientes tres acciones estructuradas:

Acción 1: Establecer un Período de 'Hipercurado' y Sombra Operativa (Hypercare & Shadowing)

El equipo de desarrollo NO se disuelve inmediatamente. Se establece un período obligatorio de 'Hipercurado' (Hypercare) de 2 a 4 semanas post-lanzamiento en producción, donde los desarrolladores trabajan codo a codo con el equipo de soporte técnico (Nivel 2/3). Durante este período, los desarrolladores atienden conjuntamente los incidentes reales de producción, capacitan al equipo de soporte en tiempo real y refactorizan el código que cause fallas recurrentes.

Acción 2: Implementar Observabilidad, SLOs/SLAs y Runbooks Ejecutables

En lugar de un manual estático en Word, el proyecto debe entregar artefactos de operación vivos:

- **Telemetría y Monitoreo:** Tableros dinámicos en tiempo real (ej. Grafana, Datadog) que permitan a soporte monitorear la salud del sistema (latencia, tasa de errores, uso de CPU/Memoria).
- **Acuerdos de Servicio (SLO/SLI):** Definición clara de Objetivos de Nivel de Servicio (SLOs) e Indicadores (SLIs) respaldados por alertas automáticas.

- **Runbooks Digitales:** Guías ejecutables de resolución de problemas (Runbooks / Playbooks) donde se detalla paso a paso qué hacer cuando salta una alerta específica en producción.

El concepto de "Presupuesto de Error" (Error Budgets): Para materializar los SLOs en la práctica, el proyecto debe adoptar el concepto de "Presupuesto de Error", pilar de la Ingeniería de Confiabilidad del Sitio (SRE) de Google. El presupuesto de error cuantifica el margen de fallos permitido antes de que los usuarios se sientan insatisfechos (por ejemplo, si el SLO de disponibilidad es 99.9%, el 0.1% restante es el presupuesto de error mensual). Esto funciona como un regulador matemático y contractual: si las caídas del sistema o los incidentes críticos agotan este presupuesto, el equipo de desarrollo tiene estrictamente prohibido lanzar nuevas funcionalidades de negocio. A partir de ese momento, el 100% del esfuerzo debe redirigirse a estabilizar el código, mejorar la infraestructura y pagar deuda técnica. Este mecanismo elimina la fricción entre desarrollo y soporte, obligando a los programadores a preocuparse activamente por la confiabilidad operativa del código que escriben.

Acción 3: Transición de la Estructura de Proyecto a Equipo de Producto Continuo

El Director del Proyecto debe gestionar la transición desde una mentalidad orientada a 'Proyectos' (con fecha de inicio y fin rígidas) hacia una estructura de 'Equipo de Producto' (Product Team). Se garantiza que al menos un núcleo del equipo de ingeniería (o un Stream-Aligned Team) permanezca asignado permanentemente al producto para liderar la evolución continua, la reducción proactiva de deuda técnica y la mejora operacional (Kaisen).

Adopción del Enfoque "Shift-Left" en Operaciones: La transición exitosa hacia un Equipo de Producto exige aplicar el principio de "Shift-Left" (desplazamiento a la izquierda) a las responsabilidades operativas. En el enfoque clásico de proyectos, la creación de manuales, la configuración del monitoreo y la planificación de contingencias se dejaban para la etapa final del

ciclo de vida (a la derecha del cronograma). Bajo este nuevo paradigma continuo, el equipo asimila que la operabilidad es un requisito no funcional crítico que se desplaza hacia las fases iniciales de diseño y codificación. Los desarrolladores deben programar la telemetría, estructurar los logs y definir las alertas de soporte desde la primera línea de código que escriben, no como una actividad residual de cierre. Esto destruye definitivamente el antipatrón de "lanzar sobre el muro", garantizando que el software nazca siendo completamente gobernable y mantenible por diseño.

Conclusión

La ingeniería moderna va mucho más allá de simplemente escribir líneas de código; se trata de asegurar que ese código sobreviva, sea sostenible y aporte valor real día tras día. El primer caso nos dio un golpe de realidad sobre cómo monitoreamos nuestro trabajo. Nos dimos cuenta de que celebrar que cumplimos perfectamente con el presupuesto y el tiempo (usando las clásicas métricas de EVM) no sirve de absolutamente nada si, al final, la aplicación está llena de errores o tarda una eternidad en llegar a los usuarios. Para saber cómo estamos realmente, necesitamos combinar ese control tradicional con métricas de flujo y estándares de alto rendimiento (como DORA). Solo así podemos ver dónde nos estamos atascando y obligarnos a integrar la calidad desde el día uno, no al final. En definitiva, el mayor aprendizaje que nos llevamos de esta unidad es la urgencia de cambiar nuestro chip mental: hay que dejar de gestionar "proyectos" que mueren al entregarse, para empezar a construir "productos". El verdadero legado de un ingeniero no es la aplicación que entregó; son las lecciones que rescató en las retrospectivas, las mejoras que dejó instaladas tras las auditorías y la tranquilidad operativa que le brindó a la empresa. Un software solo cumple su propósito cuando el equipo que se queda a cargo tiene las herramientas y el conocimiento para mantenerlo vivo, sano y evolucionando.